

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2023 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2023	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Heap	2
2. Rețele de calculatoare: Concepte de bază	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Rame concentrice	3
2. Algoritm eficient: Termeni recurență în interval	4
II. Rezolvare Titularizare Varianta 3 (12 iulie 2023)	6
SUBIECTUL I (30 de puncte)	6
1. Date de tip real (Floating Point)	6
2. Tehnologia Informației (TIC): Tabele în procesoare de text	6
SUBIECTUL al II-lea (30 de puncte)	6
1. Programare: n-ouroboros	6
2. Algoritm eficient: Sumă subset 2023	9
SUBIECTUL al III-lea - Completare pentru Model 2023 (30 de puncte)	10
1. Itemi pentru divizibilitate	10
2. Demers interdisciplinar pentru ergonomie	10
SUBIECTUL al III-lea - Completare pentru Varianta 3 2023 (30 de puncte)	11
1. Exercițiul pentru grafuri hamiltoniene	11
2. Test pentru dispozitive de intrare	11

I. Rezolvare Titularizare Model 2023

SUBIECTUL I (30 de puncte)

1. Structura de date: Heap

- **Definire:** Un heap este un arbore binar aproape complet ce respectă proprietatea de heap: cheia fiecărui nod este mai mare sau egală cu cheile fiilor săi (Max-Heap) sau mai mică ori egală cu cheile acestora (Min-Heap).
- **Reprezentare:** Se utilizează un vector H în care rădăcina este $H[1]$. Pentru un nod de pe poziția i :
 - Fiu stânga: $2i$
 - Fiu dreapta: $2i + 1$
 - Părinte: $i \div 2$
- **Inserare (Max-Heap):** Se adaugă nodul pe prima poziție liberă la sfârșitul vectorului, apoi se reface proprietatea de heap prin urcarea elementului (up-heap / sift-up) prin comparații succesive cu părintele său.
- **Eliminare maxim (rădăcină):** Se înlocuiește rădăcina cu ultimul element din heap, se decrementează dimensiunea, iar apoi se coboară elementul (down-heap / sift-down / heapify) comparându-l cu cel mai mare dintre fiii săi până când structura este validă.
- **Problemă (HeapSort):**

▸ C++:

```
#include <iostream>
using namespace std;
void heapify(int A[], int n, int i) {
    int cel_mai_mare = i;
    int l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && A[l] > A[cel_mai_mare]) cel_mai_mare = l;
    if (r < n && A[r] > A[cel_mai_mare]) cel_mai_mare = r;
    if (cel_mai_mare != i) {
        swap(A[i], A[cel_mai_mare]);
        heapify(A, n, cel_mai_mare);
    }
}
void heapSort(int A[], int n) {
    for (int i = n / 2 - 1; i >= 0; --i) heapify(A, n, i);
    for (int i = n - 1; i > 0; --i) {
        swap(A[0], A[i]);
        heapify(A, i, 0);
    }
}
int main() {
    int A[] = {12, 11, 13, 5, 6, 7};
    heapSort(A, 6);
    for (int x : A) cout << x << " ";
    return 0;
}
```

2. Rețele de calculatoare: Concepte de bază

- **Definiție:** Un grup de calculatoare conectate prin canale de comunicație pentru partajarea resurselor.
- **Avantaje:** Partajarea resurselor hardware (ex. imprimante) și software, stocare centralizată și comunicare rapidă (email, chat).
- **Tipuri:** LAN (Local Area Network), MAN (Metropolitan Area Network), WAN (Wide Area Network).
- **Protocoale:** TCP (controlul transmisiei), IP (adresa destinație).
- **Echipamente:** Switch, Router.

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Rame concentrice

Soluție C++:

```
#include <iostream>
using namespace std;

int a[51][51];

void pqrama(int a[51][51], int p, int q, int x) {
    for (int j = p; j <= q; ++j) {
        a[p][j] = x;
        a[q][j] = x;
    }
    for (int i = p; i <= q; ++i) {
        a[i][p] = x;
        a[i][q] = x;
    }
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    int K = (n + 1) / 2;
    for (int i = 1; i <= K; ++i) {
        pqrama(a, i, n - i + 1, K - i + 1);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            cout << a[i][j] << (j == n ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}
```

Soluție Pascal:

```
program RameConcentrice;
type
    TMatrice = array[1..50, 1..50] of integer;
```

```

var
  a: TMatrice;
  n, i, j, K: integer;

procedure pqrama(var a: TMatrice; p, q, x: integer);
var
  idx: integer;
begin
  for idx := p to q do
  begin
    a[p, idx] := x;
    a[q, idx] := x;
  end;
  for idx := p to q do
  begin
    a[idx, p] := x;
    a[idx, q] := x;
  end;
end;

begin
  read(n);
  K := (n + 1) div 2;
  for i := 1 to K do
    pqrama(a, i, n - i + 1, K - i + 1);

  for i := 1 to n do
  begin
    for j := 1 to n do
    begin
      write(a[i, j]);
      if j < n then write(' ');
    end;
    writeln;
  end;
end.

```

2. Algoritm eficient: Termeni recurență în interval

Metoda: Termenii șirului sunt calculați cu formula $a_n = n^2 + n + 1$ și sunt strict crescători. Generăm termenii în această ordine, până când termenul depășește y ; fiecare termen aflat în intervalul $[x, y]$ este memorat. La final îi afișăm în ordine inversă, pentru a obține ordinea descrescătoare cerută.

Eficiență: Algoritmul face doar $O(\sqrt{y})$ pași, deoarece $n^2 + n + 1 \leq y$ implică n de ordinul radicalului lui y . Nu este necesară parcurgerea tuturor valorilor din intervalul $[x, y]$. Memoria este proporțională cu numărul termenilor selectați.

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

```

```
int main() {
    long long x, y;
    if (!(cin >> x >> y)) return 0;

    vector<long long> rez;
    long long n = 0;
    while (true) {
        long long val = n * n + n + 1;
        if (val > y) break;
        if (val >= x) rez.push_back(val);
        n++;
    }

    ofstream fout("titu2022.out");
    for (int i = rez.size() - 1; i >= 0; --i) {
        fout << rez[i] << (i == 0 ? "" : " ");
    }
    fout.close();

    return 0;
}
```

Soluție Pascal:

```
program RecurentaFisier;
var
    x, y, n, val: int64;
    rez: array[1..40000] of int64;
    count, i: integer;
    fout: text;
begin
    readln(x, y);
    n := 0;
    count := 0;
    while true do
        begin
            val := n * n + n + 1;
            if val > y then break;
            if val >= x then
                begin
                    count := count + 1;
                    rez[count] := val;
                end;
            n := n + 1;
        end;

    assign(fout, 'titu2022.out');
    rewrite(fout);
    for i := count downto 1 do
        begin
            write(fout, rez[i]);
            if i > 1 then write(fout, ' ');
        end;
    close(fout);
end.
```

—

II. Rezolvare Titularizare Varianta 3 (12 iulie 2023)

SUBIECTUL I (30 de puncte)

1. Date de tip real (Floating Point)

- **Tip real in C++:** double (8 octeți / 64 biți).
- **Tip real in Pascal:** double sau real (8 octeți în Pascal modern).
- **Patru exemple de declarare cu inițializări (C++ / Pascal):**
 1. Zecimal standard: double a = 3.14; / a: double = 3.14;
 2. Partea fracționară omisă: double b = 5.; / b: double = 5.0;
 3. Partea întregă omisă: double c = .75; / c: double = 0.75;
 4. Reprezentare exponențială (științifică): double d = 1.2e-3; / d: double = 1.2e-3;
- **Exemple de operatori:** adunarea (+), împărțirea reală (/).
- **Funcții predefinite:** radical (sqrt), valoare absolută (abs), rotunjire (round).
- **Problemă (Aria unui cerc):**
 - **C++:**

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double r;
    cin >> r;
    double aria = M_PI * r * r;
    cout << aria << endl;
    return 0;
}
```

- **Pascal:**

```
program AriaCerc;
var r, aria: double;
begin
    read(r);
    aria := pi * r * r;
    writeln(aria);
end.
```

2. Tehnologia Informației (TIC): Tabele în procesoare de text

- **Noțiuni:** Celulă, linie, coloană. Tabelul se poate insera în corpul documentului sau în antet/subsol.
 - **Inserare:** Din meniul Insert (Tabel $M \times N$) sau prin desenarea manuală a tabelului.
 - **Personalizare:** Îmbinare celule (Merge), divizare (Split), culori de fundal (Shading), borduri (Borders), lățime coloană (Width), înălțime rând (Height), aliniere text în celulă.
-

SUBIECTUL al II-lea (30 de puncte)

1. Programare: n-ouroboros

Soluție C++:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <sstream>

using namespace std;

int ouroboros(string s) {
    int len = s.length();
    int max_n = 0;
    for (int n = 1; 2 * n <= len; ++n) {
        string prefix = s.substr(0, n);
        string suffix = s.substr(len - n, n);
        if (prefix == suffix) {
            max_n = n;
        }
    }
    return max_n;
}

string to_lower_str(string s) {
    string res = s;
    for (char &c : res) {
        c = tolower(c);
    }
    return res;
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string word;
    vector<string> words;
    int max_overall_n = 0;
    vector<string> best_words;

    while (ss >> word) {
        words.push_back(word);
        string lower_w = to_lower_str(word);
        int n = ouroboros(lower_w);
        if (n > max_overall_n) {
            max_overall_n = n;
            best_words.clear();
            best_words.push_back(word);
        } else if (n == max_overall_n && n > 0) {
            best_words.push_back(word);
        }
    }

    if (max_overall_n > 0) {
        cout << max_overall_n << endl;
        for (int i = 0; i < best_words.size(); ++i) {
            cout << best_words[i] << (i == best_words.size() - 1 ? "" : " ");
        }
    }
}
```

```

    }
    cout << endl;
} else {
    cout << "NU" << endl;
}

return 0;
}

```

Soluție Pascal:

```

program OuroborosText;
uses sysutils;

var
    textInput: string;
    words, best_words: array[1..100] of string;
    wordCount, bestWordCount, i, max_overall_n, n: integer;
    currWord: string;

function ouroboros(s: string): integer;
var
    len, max_n, n_val: integer;
    prefix, sufix: string;
begin
    len := length(s);
    max_n := 0;
    for n_val := 1 to len div 2 do
    begin
        prefix := copy(s, 1, n_val);
        sufix := copy(s, len - n_val + 1, n_val);
        if prefix = sufix then
            max_n := n_val;
        end;
    end;
    ouroboros := max_n;
end;

begin
    readln(textInput);
    wordCount := 0;
    bestWordCount := 0;
    max_overall_n := 0;

    // Împărțire în cuvinte
    currWord := '';
    for i := 1 to length(textInput) do
    begin
        if textInput[i] <> ' ' then
            currWord := currWord + textInput[i]
        else if currWord <> '' then
        begin
            wordCount := wordCount + 1;
            words[wordCount] := currWord;
            currWord := '';
        end;
    end;
    if currWord <> '' then

```



```

begin
    wordCount := wordCount + 1;
    words[wordCount] := currWord;
end;

for i := 1 to wordCount do
begin
    n := ouroboros(lowercase(words[i]));
    if n > max_overall_n then
    begin
        max_overall_n := n;
        bestWordCount := 1;
        best_words[1] := words[i];
    end
    else if (n = max_overall_n) and (n > 0) then
    begin
        bestWordCount := bestWordCount + 1;
        best_words[bestWordCount] := words[i];
    end;
end;

if max_overall_n > 0 then
begin
    writeln(max_overall_n);
    for i := 1 to bestWordCount do
    begin
        write(best_words[i]);
        if i < bestWordCount then write(' ');
    end;
    writeln;
end
else
    writeln('NU');
end.

```

2. Algoritm eficient: Sumă subset 2023

- **Algoritmul:** Utilizăm un vector caracteristic boolean possible de lungime 2024, inițializat cu false și possible[0] = true. Citim numerele din fișier și, pentru fiecare număr x care satisface $1 \leq x \leq 2023$, actualizăm posibilitatea sumelor de la 2023 în jos: possible[v] = possible[v] || possible[v - x]. Această abordare rulează în $O(M \times 2023)$ unde M este numărul de numere, adică extrem de rapid. Spațiul este constant ($O(1)$).

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

bool possible[2024];

int main() {
    ifstream fin("titu2023.in");
    if (!fin) { cout << "NU\n"; return 0; }
    possible[0] = true;
    int x;
    while (fin >> x) {

```

```

        if (x >= 1 && x <= 2023) {
            for (int v = 2023; v >= x; --v) {
                if (possible[v - x]) possible[v] = true;
            }
        }
    }
    fin.close();
    if (possible[2023]) cout << "DA\n";
    else cout << "NU\n";
    return 0;
}

```

Soluție Pascal:

```

program Suma2023;
var
    fin: text;
    possible: array[0..2023] of boolean;
    x, v: integer;
begin
    assign(fin, 'titu2023.in'); reset(fin);
    for v := 0 to 2023 do possible[v] := false;
    possible[0] := true;
    while not eof(fin) do
        begin
            read(fin, x);
            if (x >= 1) and (x <= 2023) then
                begin
                    for v := 2023 downto x do
                        if possible[v - x] then possible[v] := true;
                    end;
                end;
        end;
    close(fin);
    if possible[2023] then writeln('DA')
    else writeln('NU');
end.

```

SUBIECTUL al III-lea - Completare pentru Model 2023 (30 de puncte)

1. Itemi pentru divizibilitate

- **Obiectiv:** Un număr prim are exact doi divizori pozitivi. (A/F) **Răspuns:** Adevărat.
- **Semiobiectiv:** Completați: restul împărțirii lui a la b se obține cu operatorul [spațiu liber].
Răspuns: mod / %.
- **Subiectiv:** Scrieți algoritmul pentru determinarea divizorilor unui număr. **Răspuns:** parcurgere d=1..n sau până la radical cu perechi de divizori.

2. Demers interdisciplinar pentru ergonomie

Particularități: leagă TIC de sănătate și educație civică; dezvoltă conduite preventive. **Metodă:** proiectul. **Mijloc:** calculator, imagini comparative, fișă de evaluare. **Formă:** grupe.

Scenariu: Elevii identifică riscuri ale utilizării calculatorului, proiectează un poster cu reguli ergonomice și prezintă argumentat soluțiile.

SUBIECTUL al III-lea - Completare pentru Varianta 3 2023 (30 de puncte)

1. Exercițiul pentru grafuri hamiltoniene

Caracteristici: repetare conștientă, feedback imediat, gradare de la sarcini simple la sarcini complexe. **Tipuri:** exerciții de recunoaștere (identificarea unui graf hamiltonian) și exerciții aplicative (construirea/verificarea unui ciclu hamiltonian).

Mijloc de învățământ: fișă cu grafuri și tablă/proiector pentru reprezentarea muchiilor. **Formă de organizare:** frontal pentru fixarea definiției, apoi lucru pe perechi. **Activitate:** identificarea unui ciclu hamiltonian într-un graf dat. **Scenariu:** Profesorul reamintește că un ciclu hamiltonian trece o singură dată prin fiecare vârf și revine în vârful inițial. Elevii primesc trei grafuri, propun succesiuni de vârfuri și verifică existența muchiilor consecutive. Profesorul cere justificarea respingerii unei succesiuni dacă lipsește o muchie sau dacă un vârf se repetă. Elevii formulează la final criteriul de verificare.

2. Test pentru dispozitive de intrare

Test scris - 90 puncte + 10 puncte din oficiu

1. **Alegere multiplă (30p):** Dispozitiv de intrare este: A. monitor B. tastatură C. imprimantă D. boxe.
 - **Răspuns:** B.
 - **Criterii:** selectarea variantei corecte 25p; nealegerea variantelor de ieșire 5p.
2. **Răspuns scurt (30p):** Precizați rolul mouse-ului în utilizarea calculatorului personal.
 - **Răspuns:** permite introducerea comenzilor prin indicare, selectare, glisare și activare a elementelor de interfață.
 - **Criterii:** identificarea rolului de dispozitiv de intrare 10p; menționarea selectării/indicării 10p; formulare clară 10p.
3. **Întrebare structurată (30p):** Dați două exemple de dispozitive de intrare și câte o situație de utilizare pentru fiecare.
 - **Răspuns:** tastatură - introducere text; scanner - digitizarea unui document; microfon - înregistrarea vocii.
 - **Criterii:** două dispozitive corecte 10p; două situații de utilizare adecvate 14p; corelarea dispozitiv-situație și claritate 6p.